🔥 4. Strings & StringBuilder (LEETCODE HEAVY)

A. String Immutability (THIS BREAKS PEOPLE)

Problem 4.1 – Illusion of Change

Task:

```
String s = "hello";
s.toUpperCase();
System.out.println(s);
```

Question (MANDATORY):

- Why does the output not change?
- Where is the new string created?

🧠 *DSA relevance:* Many bugs come from assuming strings change in-place.

Problem 4.2 – Method Trap

Task:
Write a method:

```
void change(String s) {
    s = "world";
}
```

Call it from `main()` with `"hello"`.

Question:
Why doesn't `main()` see `"world"`?

🧠 If you say "Java is pass by reference", **you are wrong**.

B. Common String Methods (YOU MUST MASTER THESE)

Problem 4.3 – Frequency Counter

Task:
Count how many times `'a'` appears in a string.

Constraints:

- Use `charAt()`
- Use `length()`
- No extra libraries

 Foundation of most string problems.

Problem 4.4 – Reverse Without StringBuilder

Task:
Reverse a string using:

- loop
- `charAt()`

❌No `StringBuilder`

 Forces character-level thinking.

Problem 4.5 – Substring Logic

Task:
Given `"programming"`, print all substrings of length 3.

 This logic appears in sliding window problems.

Problem 4.6 – Compare Trap

Task:

```
String a = "java";
String b = new String("java");
```

Check equality using:

- `==`
- `.equals()`

Question:
Why do results differ?

 LeetCode bugs live here.

C. StringBuilder vs String (PERFORMANCE CRITICAL)

Problem 4.7 – Time Bomb

Task:
Create a string by appending numbers from 1 to 1000 using:

1. `String`
2. `StringBuilder`

Question:
Which is faster and **why exactly**?

 Saying “StringBuilder is faster” without reason = weak understanding.

Problem 4.8 – Reverse Efficiently

Task:
Reverse a string using `StringBuilder`.

Then answer:

- Why is this more memory-efficient?
- Why is this allowed while String is immutable?

D. Character Arrays (LOW-LEVEL STRING CONTROL)

Problem 4.9 – Manual Reverse

Task:
Convert string to `char[]`, reverse **in-place**, convert back.

 DSA relevance: In-place manipulation is tested heavily.

Problem 4.10 – Palindrome Check

Task:
Check if a string is palindrome using:

- `char[]`
- two-pointer technique

 This is a direct LeetCode pattern.

---

5. Basic OOP (ONLY WHAT DSA NEEDS)

You are **correct** to exclude inheritance, interfaces, polymorphism.
Anyone telling you otherwise at this stage is wasting your time.

A. Class & Object (DATA HOLDERS)

Problem 5.1 – Simple Data Container

Task:
Create a class `Pair` holding two integers.

Create an object and print values.

 DSA relevance: Used in graph edges, coordinates, intervals.

B. Constructors (INITIALIZATION MATTERS)

Problem 5.2 – Constructor Discipline

Task:
Create a class with:

- default constructor

- parameterized constructor

Print values after object creation.

Question:
Why are constructors safer than setter methods?

Problem 5.3 – Constructor Bug

Task:
Create a constructor but forget to initialize one variable.

Question:
Why does Java not warn you?

 Silent bugs are dangerous in DSA.

C. `this` Keyword (YOU WILL NEED THIS)

Problem 5.4 – Shadowing Problem

Task:

```
class Box {
    int size;
    Box(int size) {
        size = size;
    }
}
```

Question:
Why is `size` always zero?

 If you don't answer this correctly, stop and fix it.

Problem 5.5 – Correct Fix

Fix the above using `this`.

Explain **exactly** what `this` refers to.

D. Encapsulation (MINIMAL, BUT NECESSARY)

Problem 5.6 – Controlled Access

Task:
Create a class with:

- private variable
- public getter
- no setter

Question:
Why is this useful in DSA-style code?

 Immutable data prevents logic corruption.

Problem 5.7 – Mutation Risk

Task:
Expose internal array using getter.

Question:
Why is this dangerous?

 Real interview trick.

FINAL REALITY CHECK

If you think:

- Strings are pass-by-reference → **You are wrong**
- == compares string content → **You are wrong**
- OOP theory is more important than usage → **You are wrong**

Correct thinking:

- Java is **pass-by-value**
- Strings are **immutable objects**

- OOP in DSA is for **structuring data, not philosophy**